LAPORAN PRATIKUM

STRUKTUR DATA

“Pratikum Pekan 1”

Disusun Oleh:

Rafikhul Ramadhan

2511533012

Dosen Pengampu : Dr. Wahyudi, S.T, M.T.

Asisten Praktikum : Muhammad Galis Avero

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Segala puji dan syukur saya panjatkan ke hadirat Allah SWT atas limpahan rahmat, taufik, dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur Data ini dengan baik dan tepat waktu.

Laporan praktikum ini disusun sebagai salah satu bentuk pemenuhan tugas akademik, sekaligus sebagai sarana untuk memperdalam pemahaman penulis terhadap konsep-konsep dasar dalam struktur data. Melalui praktikum ini, saya mempelajari berbagai materi penting seperti penggunaan array, stack, queue, linked list, serta bagaimana mengimplementasikannya dalam bahasa pemrograman. Selain itu, kegiatan praktikum juga melatih kemampuan berpikir logis, sistematis, dan terstruktur dalam menyelesaikan permasalahan.

Saya menyadari bahwa dalam penyusunan laporan ini masih terdapat berbagai kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, saya sangat mengharapkan kritik dan saran yang membangun guna perbaikan di masa yang akan datang.

Pada kesempatan ini, saya juga ingin menyampaikan rasa terima kasih kepada dosen pengampu, asisten praktikum, serta teman-teman yang telah memberikan bimbingan, dukungan, dan bantuan selama proses praktikum hingga penyusunan laporan ini.

Akhir kata, semoga laporan praktikum ini dapat memberikan manfaat bagi penulis khususnya, serta bagi para pembaca pada umumnya. Penulis berharap laporan ini juga dapat digunakan sebagai referensi dalam memahami materi praktikum, baik pada mata kuliah Struktur Data maupun praktikum lainnya.

.

Padang, 1 April 2026

Rafikhul Ramadhan

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 1
BAB II PEMBAHASAN 2
2.1 Langkah Kerja 2
2.2 Contoh Program Char ..... **Error! Bookmark not defined.**
2.3 Deklarasi Variabel ..... **Error! Bookmark not defined.**
2.4 Keliling Lingkaran ..... **Error! Bookmark not defined.**
BAB III KESIMPULAN 9
3.1 Ringkasan 9
3.2 Saran ..... **Error! Bookmark not defined.**
DAFTAR PUSTAKA 10

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam perkembangan teknologi informasi, pemahaman tentang struktur data dan pemrograman berorientasi objek menjadi hal yang sangat penting bagi seorang programmer. Struktur data digunakan untuk mengelola dan menyimpan data secara efisien, sedangkan konsep objek membantu dalam memodelkan permasalahan ke dalam bentuk program.

Pada praktikum ini, saya mempelajari implementasi kelas dan objek melalui pembuatan program yang merepresentasikan waktu (jam). Program tersebut mencakup berbagai operasi seperti validasi input, konversi ke detik, perbandingan waktu, serta manipulasi data seperti penjumlahan dan pengurangan waktu. Dengan praktikum ini, diharapkan penulis dapat memahami cara kerja objek serta penerapannya dalam pemrograman.

1.2 Tujuan

Tujuan dari praktikum ini adalah untuk memahami konsep dasar struktur data dan pemrograman berorientasi objek, serta mampu mengimplementasikan kelas dan objek dalam bentuk program. Selain itu, praktikum ini juga bertujuan untuk melatih kemampuan dalam membuat operasi-operasi dasar terhadap suatu data secara sistematis dan terstruktur.

1.3 Manfaat

Manfaat dari praktikum ini adalah meningkatkan pemahaman penulis terhadap konsep struktur data dan objek dalam pemrograman. Selain itu, praktikum ini juga membantu dalam melatih kemampuan berpikir logis, sistematis, dan terstruktur dalam menyelesaikan suatu permasalahan. Hasil dari praktikum ini diharapkan dapat menjadi dasar dalam pengembangan program yang lebih kompleks di masa mendatang.

BAB II

PEMBAHASAN

2.1 Langkah Kerja

Tahapan yang dilakukan dalam pratikum ini adalah sebagai berikut:

1. Menjalankan aplikasi IDE (seperti Eclipse atau IntelliJ IDEA) atau menggunakan text editor yang mendukung bahasa Java.
2. Membuat sebuah project baru dengan nama **pekan1**.
3. Menambahkan tiga buah file Java, yaitu Jam_2511533012.java, JamDriver_2511533012.java, dan JamDriver2_2511533012.java.
4. Menuliskan kode program sesuai dengan instruksi yang diberikan pada masing-masing file.
5. Melakukan proses kompilasi menggunakan *Java compiler* agar program dapat dijalankan.
6. Menjalankan program untuk mengamati hasil keluaran sesuai dengan logika yang dibuat.
7. Mengevaluasi hasil eksekusi pada *console* serta menarik kesimpulan dari setiap percobaan yang dilakukan.

2.2 ADT (Jam_2511533012.java

```java
package pekan1_2511533012;

public class Jam_2511533012 {
	private int hh;
	private int mm;
	private int ss;

	public static boolean isValid(int h, int m, int s) {
		return (0 <= h && h <= 23 ) && (0 <= m && m <= 59) &&
		(0 <= s && s <= 59);
	}

	public Jam_2511533012(int h, int m, int s) {
		this.hh = h; this.mm = m; this.ss = s;
	}
```

```java
    public int getHour() {
        return hh;
    }
    public int getMinute() { return mm; }
    public int getSecond() { return ss; }

    public void setHour(int h) { this.hh = h; }
    public void setMinute(int m) { this.mm = m; }
    public void setSecond(int s) { this.ss = s; }

    public int toSeconds() {
        return hh * 3600 + mm * 60 + ss;
    }
    public static Jam_2511533012 fromSeconds(int total) {
        if (total < 0) throw new
    IllegalArgumentException("detiki negatif");
        total %= 24 * 3600;
        int h = total / 3600; total %= 3600;
        int m = total / 60; int s = total % 60;
        return new Jam_2511533012(h, m, s);
    }

    public int compareTo(Jam_2511533012 other) {
        return Integer.compare(this.toSeconds(),
    other.toSeconds());
    }
    public boolean equals(Object o) {
        if(!(o instanceof Jam_2511533012 j)) return false;
        return hh == j.hh && mm == j.mm && ss == j.ss;
    }
    public int hashCode() {
        return java.util.Objects.hash(hh, mm, ss);
    }

    public Jam_2511533012 plus(Jam_2511533012 other) {
        return fromSeconds(this.toSeconds() +
        other.toSeconds());
    }
    public Jam_2511533012 minus(Jam_2511533012 other) {
        return fromSeconds(Math.floorMod(this.toSeconds() -
        other.toSeconds(), 24 * 3600));
    }
    public Jam_2511533012 nextSecond() {
        return fromSeconds(this.toSeconds() + 1);
    }
    public Jam_2511533012 nextNSeconds(int n) {
        return fromSeconds(this.toSeconds() + Math.max(0, n));
    }
    public Jam_2511533012 prevSecond() {
        return fromSeconds(Math.floorMod(this.toSeconds() - 1,
        24 * 3600));
    }
    public Jam_2511533012 prevNSeconds(int n) {
```

```
            return fromSeconds (Math.floorMod(this.toSeconds() -
            Math.max(0, n), 24 * 3600));
        }

        public static int durasiDetik(Jam_2511533012 jaw,
        Jam_2511533012 jakh) {
            return jakh.toSeconds() - jaw.toSeconds();
        }

        public String toString() {
            return String.format("&02d:%02d:%02d", hh, mm, ss);
        }

}
```

2.3 JamDriver_2511533012

```
package pekan1_2511533012;

public class JamDriver_2511533012 {

        public static void main(String[] args) {
            // TODO Auto-generated method stub
            Jam_2511533012 a = new Jam_2511533012(23, 59, 50);
            Jam_2511533012 b = new Jam_2511533012(0, 0, 15);
            System.out.println("a                    = " + a);
            System.out.println("b                    = " + b);
            System.out.println("a+b                  = " +
            a.plus(b));
            System.out.println("next 20s             = " +
            a.nextNSeconds(20));
            System.out.println("durasi (a,b) = " +
            Jam_2511533012.durasiDetik(a, b));
            System.out.println("a.compareTo(b) = " +
a.compareTo(b) );
        }

}
```

2.4 JamDriver2_2511533012

```
package pekan1_2511533012;
import java.util.Scanner;
public class JamDriver2_2511533012 {

	public static void main(String[] args) {
		// TODO Auto-generated method stub
		Scanner input = new Scanner (System.in);
		System.out.println("=== Program Driver Objek Jam ===");

		System.out.println("\n--- Input Jam 1 ---");
		Jam_2511533012 j1 = buatJamDariInput(input);

		System.out.println("\n--- Input Jam 2 ---");
		Jam_2511533012 j2 = buatJamDariInput(input);

		System.out.println("\n--- Hasil Operasi ---");
		System.out.println("Jam 1 (string) : " +
j1.toString());
		System.out.println("Jam 2 (string) : " +
j2.toString());
		System.out.println("Jam 1 dalam detik : " +
j1.toSeconds());
		System.out.println("Jam 2 dalam detik : " +
j2.toSeconds());

		int perbandingan = j1.compareTo(j2);
		if (perbandingan > 0) {
			System.out.println("Status : Jam 1 lebih dalmbat
(setelah) Jam 2");
		} else if (perbandingan < 0 ) {
			System.out.println("Status : Jam 1 lebih awal
(sebelum) Jam 2");
		} else {
			System.out.println("Status : Jam 1 dan Jam 2 sama
persis");
		}

		System.out.println("Durasi (j1 ke j2) : " +
Jam_2511533012.durasiDetik(j1, j2) + " detik");

		Jam_2511533012 jNext = j1.nextSecond();
		System.out.println("Jam 1 detik berikutnya : " +
jNext);

		Jam_2511533012 jPrev = j1.prevSecond();
		System.out.println("Jam 1 detik sebelumnya : " +
jPrev);

		Jam_2511533012 jHasilPlus = j1.plus(j2);
		System.out.println("Hasil j1 + j2 : " + jHasilPlus);
```

```java
            input.close();
            System.out.println("\nProgram Selesai");
        }

        private static Jam_2511533012 buatJamDariInput(Scanner sc) {
            int h, m, s;
            while (true) {
                System.out.print("Masukkan Jam (0 - 23) : ");
                h = sc.nextInt();
                System.out.print("Masukkan Menit (0 - 59) : ");
                m = sc.nextInt();
                System.out.print("Masukkan Detik (0 - 59) : ");
                s = sc.nextInt();

                if(Jam_2511533012.isValid(h, m, s) ) {
                    return new Jam_2511533012(h, m, s);
                } else {
                    System.out.println("[ERROR] Input tidak
                    valid! silahkan di ulang.\n");
                }
            }
        }

}
```

Output dan Analisis dari semua syntax (**2.1. Jam_2511533012, 2.2. JamDriver_2511533012, JamDriver2_2511533012)**

JamDriver_2511533012:

```
<terminated> JamDriver_2511533012 [Java Application] C:\Pr
a                        = &02d:23:59
b                        = &02d:00:00
a+b                      = &02d:00:00
next 20s         = &02d:00:00
durasi (a,b) = -86375
a.compareTo(b) = 1
```

JamDriver2_2511533012:

```
<terminated> JamDriver2_2511533012 [Java Application] C:\Program Files\Java\jdk-24\bin\jav
=== Program Driver Objek Jam ===

--- Input Jam 1 ---
Masukkan Jam (0 - 23) : 12
Masukkan Menit (0 - 59) : 0
Masukkan Detik (0 - 59) : 0

--- Input Jam 2 ---
Masukkan Jam (0 - 23) : 6
Masukkan Menit (0 - 59) : 0
Masukkan Detik (0 - 59) : 0

--- Hasil Operasi ---
Jam 1 (string) : &02d:12:00
Jam 2 (string) : &02d:06:00
Jam 1 dalam detik : 43200
Jam 2 dalam detik : 21600
Status : Jam 1 lebih dalmbat (setelah) Jam 2
Durasi (j1 ke j2) : -21600 detik
Jam 1 detik berikutnya : &02d:12:00
Jam 1 detik sebelumnya : &02d:11:59
Hasil j1 + j2 : &02d:18:00

Program Selesai
```

Analisis

Program terdiri dari tiga bagian utama, yaitu kelas Jam_2511533012 sebagai inti (model data), serta dua driver (JamDriver dan JamDriver2) sebagai penguji program.

Saat program dijalankan, kelas **Jam_2511533012** berfungsi untuk merepresentasikan waktu dalam bentuk jam, menit, dan detik. Kelas ini memiliki berbagai method seperti toSeconds() untuk mengubah waktu ke detik, compareTo() untuk membandingkan dua waktu, plus() untuk menjumlahkan waktu, serta nextSecond() dan prevSecond() untuk mengetahui waktu berikutnya atau sebelumnya. Selain itu, terdapat validasi input menggunakan method isValid() agar data yang dimasukkan sesuai dengan format waktu yang benar.

Pada **JamDriver_2511533012**, program langsung menggunakan data yang sudah ditentukan (hardcoded). Output yang dihasilkan menampilkan nilai waktu, hasil penjumlahan dua objek waktu, penambahan beberapa detik, durasi antar waktu, serta hasil perbandingan. Dari sini dapat dilihat bahwa operasi-operasi dalam kelas Jam berjalan dengan baik tanpa input dari user.

Sedangkan pada **JamDriver2_2511533012**, program bersifat interaktif karena menerima input dari pengguna. Program akan meminta pengguna memasukkan

jam, menit, dan detik, kemudian melakukan validasi. Jika input tidak valid, maka pengguna diminta mengulang. Setelah input benar, program menampilkan berbagai hasil seperti konversi ke detik, perbandingan dua waktu, durasi, serta operasi penjumlahan dan perubahan waktu (next dan previous second).

Secara keseluruhan, setelah dijalankan, program menunjukkan bahwa konsep **pemrograman berorientasi objek** telah diterapkan dengan baik, di mana data dan operasi dibungkus dalam satu kelas. Program juga mampu menangani input, melakukan perhitungan waktu, serta memberikan output yang sesuai dengan fungsi yang dibuat.

.

BAB III
KESIMPULAN

3.1 Ringkasan

Pada praktikum ini, penulis mempelajari implementasi konsep pemrograman berorientasi objek melalui pembuatan kelas Jam yang merepresentasikan waktu dalam bentuk jam, menit, dan detik. Program dilengkapi dengan berbagai method seperti validasi input, konversi waktu ke detik, perbandingan dua objek waktu, serta operasi manipulasi waktu seperti penjumlahan dan perubahan detik. Pengujian dilakukan menggunakan dua driver, yaitu program dengan data langsung dan program berbasis input pengguna, sehingga menunjukkan bahwa seluruh fungsi dalam kelas dapat berjalan dengan baik.

3.2 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa konsep struktur data dan pemrograman berorientasi objek dapat diterapkan dengan baik dalam pembuatan program sederhana. Penggunaan kelas dan method mempermudah dalam pengelolaan data dan operasi yang dilakukan. Selain itu, praktikum ini juga membantu meningkatkan pemahaman dalam berpikir logis, sistematis, serta memberikan dasar yang kuat untuk pengembangan program yang lebih kompleks di masa depan..

DAFTAR PUSTAKA

[1] Oracle, "The Java Tutorials," 2023. [Online]. Available: https://docs.oracle.com/javase/tutorial/.

[2] H. M. Deitel dan P. J. Deitel, Java: How to Program, 10th ed. Boston: Pearson, 2015

Tugas Pratikum

1. Mobil_2511533012.java

```
package pekan1_2511533012;

public class Mobil_2511533012 {
    private String nama;
    private int tahun;
    private int cc;
    private double harga;
    private String merk;

    public Mobil_2511533012(String nama, int tahun, int cc, double
harga, String merk) {
        this.nama = nama;
        this.tahun = tahun;
        this.cc = cc;
        this.harga = harga;
        this.merk = merk;
    }

    public String getNama() {
         return nama;
    }
    public int getTahun() {
         return tahun;
    }
    public int getCc() {
         return cc;
    }
    public double getHarga() {
         return harga;
    }
    public String getMerk() {
         return merk;
    }

    public void setNama(String nama) {
         this.nama = nama;
    }
    public void setTahun(int tahun) {
         this.tahun = tahun;
    }
    public void setCc(int cc) {
         this.cc = cc;
    }
    public void setHarga(double harga) {
         this.harga = harga;
    }
    public void setMerk(String merk) {
         this.merk = merk;
```

```
    }

    public String toString() {
        return nama + ", " + tahun + ", " + cc + "cc, Rp" + harga + ",
" + merk;
    }
}
```

2. MobilDriver_2511533012.java

```
package pekan1_2511533012;

public class MobilDriver_2511533012 {

    public static void main(String[] args) {

        Mobil_2511533012[] dataMobil = new Mobil_2511533012[10];
        int jumlah = 0;

        dataMobil[jumlah++] = new Mobil_2511533012("Avanza", 2020,
1300, 150000000, "Toyota");
        dataMobil[jumlah++] = new Mobil_2511533012("Brio", 2022, 1200,
180000000, "Honda");

        System.out.println("=== Data Mobil ===");
        tampil(dataMobil, jumlah);

        int indexHapus = 0;
        for (int i = indexHapus; i < jumlah - 1; i++) {
            dataMobil[i] = dataMobil[i + 1];
        }
        jumlah--;

        System.out.println("\n=== Setelah Hapus ===");
        tampil(dataMobil, jumlah);

        dataMobil[0].setHarga(175000000);

        System.out.println("\n=== Setelah Update ===");
        tampil(dataMobil, jumlah);

        System.out.println("\n=== Ambil Data (Getter) ===");
        System.out.println("Nama mobil: " + dataMobil[0].getNama());
    }

    public static void tampil(Mobil_2511533012[] data, int jumlah) {
        for (int i = 0; i < jumlah; i++) {
            System.out.println(data[i]);
        }
    }
}
```

3. **Output dan Penjelasan Syntax**

```
=== Data Mobil ===
Avanza, 2020, 1300cc, Rp1.5E8, Toyota
Brio, 2022, 1200cc, Rp1.8E8, Honda

=== Setelah Hapus ===
Brio, 2022, 1200cc, Rp1.8E8, Honda

=== Setelah Update ===
Brio, 2022, 1200cc, Rp1.75E8, Honda

=== Ambil Data (Getter) ===
Nama mobil: Brio
```

Penjelasan:

Program ini dibuat untuk mengimplementasikan konsep **Abstract Data Type (ADT)** pada data mobil dengan menggunakan **array manual** sebagai media penyimpanan data. Program terdiri dari dua kelas, yaitu kelas Mobil sebagai ADT dan kelas MobilDriver sebagai pengendali jalannya program.

Pada kelas Mobil, terdapat beberapa atribut yaitu nama, tahun, cc, harga, dan merk yang merepresentasikan data sebuah mobil. Kelas ini juga dilengkapi dengan **konstruktor** untuk inisialisasi objek, **getter (selektor)** untuk mengambil nilai atribut, serta **setter (mutator)** untuk mengubah nilai atribut. Selain itu, terdapat method toString() yang digunakan untuk menampilkan informasi mobil dalam bentuk yang lebih mudah dibaca.

Pada kelas MobilDriver, digunakan **array manual** dengan kapasitas tetap untuk menyimpan beberapa objek mobil. Variabel jumlah digunakan untuk mencatat banyaknya data mobil yang tersimpan dalam array. Proses **penambahan data** dilakukan dengan memasukkan objek baru ke dalam array pada indeks tertentu, kemudian nilai jumlah ditambah. Proses **penghapusan**

data dilakukan dengan cara menggeser elemen array ke kiri untuk menutupi data yang dihapus, kemudian jumlah data dikurangi.

Program juga menunjukkan penggunaan **mutator** dengan mengubah nilai atribut (misalnya harga mobil) menggunakan setter, serta penggunaan **selektor** untuk mengambil dan menampilkan data tertentu menggunakan getter. Untuk menampilkan seluruh data mobil, digunakan perulangan yang mengakses setiap elemen dalam array.

Secara keseluruhan, program ini menunjukkan bagaimana konsep ADT dapat diimplementasikan dengan array manual, serta bagaimana operasi dasar seperti tambah, hapus, akses, dan ubah data dapat dilakukan secara terstruktur dalam pemrograman Java.